

Linux for Interviews & Real-World Debugging

From Boot Process → Production Troubleshooting

- **What interviewers actually ask**
- **Real-world scenarios (not theory)**
- **Commands + debugging mindset**

Built for DevOps / SRE / Backend roles

Topic 1: Linux Boot Process

BIOS → UEFI → GRUB → Kernel → systemd

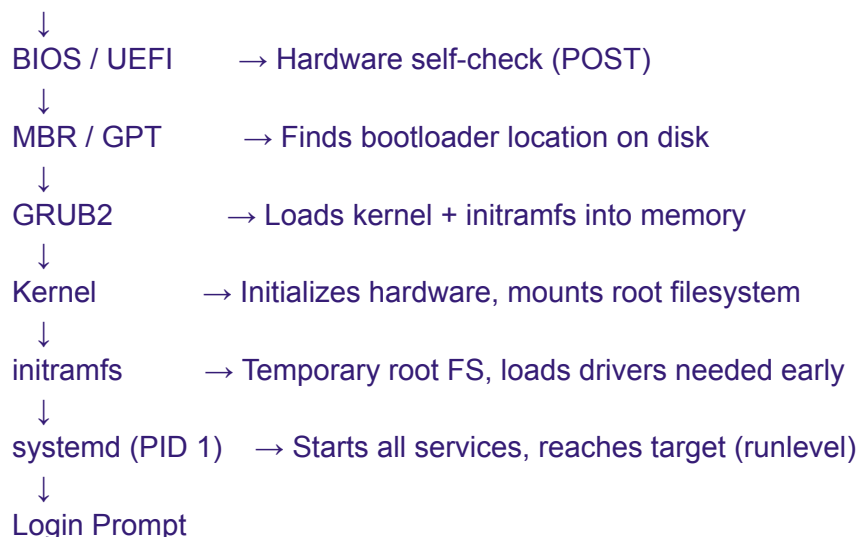
1. Core Concept

When you press the power button, the system goes through a **fixed sequence** to hand control from hardware → firmware → bootloader → kernel → init system.

Every stage has one job: **prepare and hand off to the next stage.**

2. Step-by-Step Flow

Power ON



3. Interview Questions & Answers

Q1. What is the difference between BIOS and UEFI?

BIOS is legacy firmware using MBR (max 2TB disk, 4 primary partitions). UEFI is modern, uses GPT (supports 9.4ZB disks, 128 partitions), supports Secure Boot, has faster POST, and can boot from larger partitions. Most production servers today use UEFI.

Q2. What is initramfs and why is it needed?

The kernel can't mount the real root filesystem if it doesn't have the drivers for it yet (e.g., LVM, LUKS encryption, NVMe). initramfs is a temporary RAM-based

root FS that contains just enough drivers/tools to mount the real / and hand off control to systemd.

Q3. What is PID 1 and why does it matter?

PID 1 is the first process started by the kernel — it's the parent of all processes. On modern Linux, this is **systemd**. If PID 1 dies, the kernel panics. It's responsible for starting/stopping all services and handling orphaned processes.

Q4. Server is stuck at boot — how do you troubleshoot?

1. Check GRUB menu — can you get a shell? (e to edit, add `rd.break` or `init=/bin/bash`)
 2. Boot into rescue/emergency target
 3. Check `journalctl -b` for failed units
 4. Look for filesystem errors — kernel may have dropped to emergency shell
 5. Check `dmesg` for hardware/driver failures
-

Q5. What is the difference between `multi-user.target` and `graphical.target`?

`multi-user.target` = CLI only, all network services running (equivalent to old runlevel 3). `graphical.target` = same + display manager/GUI (runlevel 5). Production servers almost always use `multi-user.target`.

4. Real-World Scenario

Scenario: A production server rebooted after a kernel update and never came back online.

What you do:

1. Access console (IPMI/iDRAC/iLO — out-of-band management)
 2. Check if it's stuck at GRUB (corrupt grub.cfg after update?)
 3. Boot into previous kernel from GRUB menu
 4. Once up, run `journalctl -b -1` to see what failed last boot
 5. Run `systemd-analyze blame` to catch any service hanging on startup
 6. If filesystem corruption → `fsck` from rescue mode
-

Topic 2: Linux Filesystem Hierarchy (FHS)

Understanding What Lives Where & Why

1. Core Concept

The **Filesystem Hierarchy Standard (FHS)** defines the directory structure of Linux. Every directory has a **specific, fixed purpose** — not arbitrary. In interviews and production, knowing *why* something lives in `/var` vs `/etc` vs `/tmp` separates juniors from seniors.

2. Step-by-Step Breakdown

/	← Root of everything. The top.
bin/	← Essential user binaries (ls, cp, mv)
sbin/	← System/admin binaries (fdisk, ifconfig)
etc/	← All config files live here
var/	← Variable data (logs, spool, databases)
tmp/	← Temporary files (cleared on reboot)
home/	← User home directories
root/	← root user's home (NOT inside /home)
lib/	← Shared libraries for /bin and /sbin
usr/	← Secondary hierarchy (most software installs here)
bin/	← Non-essential user binaries
sbin/	← Non-essential system binaries
lib/	← Libraries for /usr/bin
local/	← Manually compiled/installed software
opt/	← Optional/3rd party software (Java, Oracle, agents)
proc/	← Virtual FS — live kernel/process info
sys/	← Virtual FS — kernel hardware/device info
dev/	← Device files (disks, terminals, null)
mnt/	← Temporary manual mount points
media/	← Auto-mounted removable media (USB, CD)
boot/	← Kernel, GRUB, initramfs files
run/	← Runtime data (PID files, sockets) since last boot
srv/	← Data served by system (FTP, HTTP roots)

3. Directory Deep-Dive — The Ones That Matter Most

`/etc` — The Brain of Configuration

bash

<code>/etc/passwd</code>	# User account info (NO passwords stored here)
<code>/etc/shadow</code>	# Hashed passwords (root readable only)
<code>/etc/fstab</code>	# Filesystem mount table (critical — wrong entry = no boot)
<code>/etc/hosts</code>	# Static hostname → IP mappings

```
/etc/resolv.conf # DNS server config
/etc/sudoers     # Who can run sudo (edit only with visudo!)
/etc/ssh/sshd_config # SSH daemon config
/etc/crontab     # System-wide cron jobs
/etc/systemd/    # systemd unit overrides
```

/var — The Logbook

```
bash
/var/log/      # All system logs
/var/log/syslog # General system log (Debian/Ubuntu)
/var/log/messages # General system log (RHEL/CentOS)
/var/log/auth.log # Auth/SSH login attempts
/var/log/dmesg  # Kernel boot messages
/var/spool/cron/ # User crontabs stored here
/var/lib/       # Persistent app data (MySQL, Docker, rpm db)
/var/run/ → /run # Symlinked to /run (modern systems)
```

/dev — Everything is a File

```
bash
/dev/sda      # First SCSI/SATA disk
/dev/sda1     # First partition of sda
/dev/nvme0n1   # NVMe disk
/dev/null     # Black hole — discard output
/dev/zero     # Infinite stream of zeros
/dev/random   # Random data source
/dev/tty      # Current terminal
/dev/pts/     # Pseudo-terminals (SSH sessions)
```

/tmp vs /var/tmp — Know the Difference

	/tmp	/var/tmp
Cleared	On every reboot	Survives reboots
Use for	Short-lived temp files	Longer-lived temp data
Permission	Sticky bit set (1777)	Sticky bit set (1777)

4. Interview Questions & Answers

Q1. What is the difference between **/bin and **/usr/bin**?**

`/bin` contains **essential** binaries needed in single-user mode or during early boot (before `/usr` is mounted) — like `ls`, `cp`, `bash`. `/usr/bin` contains non-essential user binaries available after full boot. On modern systemd distros, `/bin` is a symlink to `/usr/bin` — they're merged.

Q2. `/var/log` is full and the server is throwing errors. What do you do?

1. `df -h` — confirm which partition is full
 2. `du -sh /var/log/*` — find the largest log file
 3. **Don't just delete** — truncate it: `> /var/log/bigfile.log`
 4. Check why it grew: misconfigured app, log rotation broken?
 5. Fix `logrotate` config: `/etc/logrotate.conf` or `/etc/logrotate.d/`
 6. Long term: mount `/var/log` on a **separate partition**
-

Q3. What is the difference between `/proc` and `/sys`?

`/proc` exposes **process and kernel runtime info** (CPU, memory, running processes). `/sys` (sysfs) exposes **hardware and device info** in a structured way — used by kernel to represent devices, drivers, and kernel subsystems. `/proc` is older; `/sys` is the modern replacement for hardware-related tuning.

Q4. Why is `/etc/fstab` critical? What happens if it's wrong?

`fstab` tells the kernel what filesystems to mount at boot and where. A wrong entry (bad UUID, wrong options) will cause the system to **drop into emergency shell** at boot — unable to mount root or critical partitions. Always validate with `mount -a` before rebooting after edits.

Q5. What is the sticky bit and where is it used?

Sticky bit on a **directory** means only the **file owner** can delete their own files, even if others have write permission on the directory. Classic example: `/tmp` (permissions `1777`). Without it, any user could delete other users' temp files.

5. Real-World Scenario

Scenario: Disk at 100% on a production web server. App is throwing "No space left on device."

Exact steps taken:

```
bash
df -h          # Identify full partition → /var is 100%
du -sh /var/*  # /var/log is 48GB
du -sh /var/log/* # nginx/access.log = 46GB (logrotate was broken)

# Safe truncation (don't delete — process holds file handle)
> /var/log/nginx/access.log

# Verify space freed
df -h

# Fix logrotate
vim /etc/logrotate.d/nginx
logrotate -f /etc/logrotate.d/nginx

# Long-term: check inode exhaustion too
df -i          # Sometimes inodes run out before disk space
```

Topic 3: Process Management

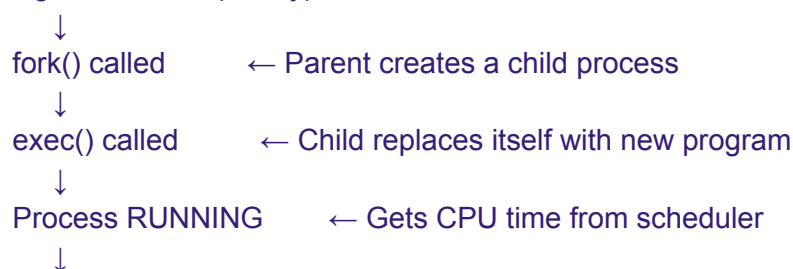
ps, top, signals, states, zombies & more

1. Core Concept

A **process** is a running instance of a program. Linux tracks every process with a **PID** (Process ID). The kernel schedules, prioritizes, and manages process lifecycle. Interviews go deep here — signals, zombie processes, orphans, and **nice** values are all fair game.

2. Step-by-Step Flow — Process Lifecycle

Program on Disk (binary)



Possible States:

└─ Running (R)	← Actively on CPU
└─ Sleeping (S)	← Waiting for I/O (interruptible)
└─ Disk Sleep (D)	← Waiting for I/O (uninterruptible) ← DANGER
└─ Stopped (T)	← Paused via SIGSTOP or Ctrl+Z
└─ Zombie (Z)	← Finished, parent hasn't read exit code
└─ Dead (X)	← Being removed from process table

↓

Process exits

↓

Parent calls wait() ← Reaps the child, clears zombie

3. Key Commands & Examples

ps — Snapshot of Processes

bash

Most used in interviews — BSD style

ps aux

a = all users, u = user-oriented format, x = include no-TTY procs

Output columns: USER PID %CPU %MEM VSZ RSS TTY STAT START TIME CMD

Show process tree (parent-child relationships)

ps auxf

Show specific process

ps aux | grep nginx

Show process with PID

ps -p 1234 -f

Show all threads of a process

ps -eLf | grep nginx

top / htop — Live Process Viewer

bash

top # Live view, refreshes every 3s

Inside top — KEY shortcuts (asked in interviews):

k → kill a process by PID

r → renice (change priority)

1 → show per-CPU breakdown

M → sort by memory

P → sort by CPU

f → add/remove columns

u → filter by user
q → quit

htop # Better UI, mouse support, easier navigation

Signals — The Most Asked Topic

bash

List all signals

kill -l

Send signal to PID

kill -SIGNAL PID

Most important signals:

kill -1 PID # SIGHUP → Reload config (no restart)

kill -2 PID # SIGINT → Interrupt (same as Ctrl+C)

kill -9 PID # SIGKILL → Force kill (cannot be caught/ignored)

kill -15 PID # SIGTERM → Graceful terminate (DEFAULT signal)

kill -19 PID # SIGSTOP → Pause process (cannot be caught)

kill -18 PID # SIGCONT → Resume paused process

Kill by name (kills all matching)

killall nginx

Kill by name, safer (matches exact binary path)

pkill -f "python app.py"

Kill all processes of a user

pkill -u username

Process Priority — nice & renice

bash

nice value range: -20 (highest priority) to +19 (lowest priority)

Default nice value = 0

Start a process with lower priority

nice -n 10 ./backup.sh

Start with higher priority (requires root)

nice -n -5 ./critical.sh

Change priority of running process

renice -n 5 -p 1234 # Lower priority of PID 1234

renice -n -5 -p 1234 # Raise priority (root only)

Check nice value

```
ps -o pid,ni,comm -p 1234
```

4. Interview Questions & Answers

Q1. What is a Zombie process? How do you remove it?

A zombie is a process that has **finished executing** but its **parent hasn't called `wait()`** to read its exit status — so the entry stays in the process table. It consumes no CPU/memory but takes a PID slot.

You cannot kill a zombie — it's already dead. Fix:

1. Find its PPID: `ps -o ppid= -p <zombie_pid>`
 2. Send SIGCHLD to parent: `kill -17 <ppid>` (asks parent to reap)
 3. If parent ignores it — kill the parent. init/systemd will reap orphans automatically.
 4. If zombies accumulate — it's a **bug in the parent application**.
-

Q2. What is the difference between `kill -9` and `kill -15`?

SIGTERM (15) is a **polite request** to terminate — the process can catch it, clean up, close connections, flush buffers, then exit gracefully. Always try this first.

SIGKILL (9) is handled by the **kernel directly** — the process never sees it, cannot catch it, cannot ignore it. No cleanup happens. Use only when SIGTERM fails. Risk: data corruption, locked files, orphaned children.

Q3. What is a process in **D** state and why can't you kill it?

D (Uninterruptible Sleep) means the process is **waiting for I/O** (usually disk or NFS) at a point in kernel code that cannot be safely interrupted. The kernel ignores all signals including SIGKILL while in this state.

Common causes: **hung NFS mount**, **dying storage**, slow disk I/O. Fix: resolve the underlying I/O issue (unmount hung NFS, fix storage). Sometimes a reboot is the only option.

Q4. What is the difference between `fork()` and `exec()`?

`fork()` creates a **copy of the current process** (child gets same memory, FDs). Both parent and child continue from that point.

`exec()` **replaces** the current process image with a new program — same PID, but entirely new code. Together, `fork()` + `exec()` is how every new process is created in Linux. (Exception: `vfork()`, `clone()` for threads)

Q5. How do you find which process is consuming the most CPU/memory?

```
bash
# Top CPU consumers
ps aux --sort=-%cpu | head -10

# Top memory consumers
ps aux --sort=-%mem | head -10

# One-liner with top (batch mode, 1 iteration)
top -b -n1 | head -20
```

6. Real-World Scenario

Scenario: Production app server is at 100% CPU. Users are reporting timeouts.

Exact steps:

```
bash
# Step 1: Identify the culprit
top                # Sort by CPU (P key), note PID

# Step 2: Investigate the process
ps -f -p <PID>      # Who owns it, full command
cat /proc/<PID>/cmdline # Exact command used to start it

# Step 3: Check if it's stuck (D state)?
ps aux | grep <PID>  # Look at STAT column

# Step 4: Check what it's doing
strace -p <PID>      # Trace system calls live
lsof -p <PID>        # What files/sockets it has open

# Step 5: Graceful fix attempt
kill -15 <PID>       # Try SIGTERM first

# Step 6: If unresponsive after 30s
kill -9 <PID>        # Force kill

# Step 7: Root cause
journalctl -u servicename # Check service logs
```

Topic 4: File Permissions & ACLs

chmod, chown, SUID, SGID, Sticky Bit, ACLs

1. Core Concept

Every file/directory in Linux has an **owner**, a **group**, and a set of **permissions** for 3 entities: owner, group, others. Permissions control **read**, **write**, **execute** access. Beyond basic permissions, **ACLs** allow fine-grained access for multiple users/groups — critical in enterprise environments.

2. Permission Structure — Read This Like a Pro

- rwx rw- r--

↑ ↑ ↑ ↑

Others (everyone else)

Group

Owner (User)

File type:

- = regular file
- d = directory
- l = symlink
- b = block device
- c = character device
- s = socket
- p = named pipe

Permission Values

r = 4 (read)
w = 2 (write)
x = 1 (execute)

$rw x = 4 + 2 + 1 = 7$
 $rw - = 4 + 2 + 0 = 6$
 $r - - = 4 + 0 + 0 = 4$
 $- - - = 0$

Common combos:

777 → rwxrwxrwx (dangerous — never in prod)

755 → rwxr-xr-x (standard for binaries/dirs)

644 → rw-r--r-- (standard for files)

600 → rw----- (private files, SSH keys)
400 → r----- (read-only, e.g. .pem files)

3. Step-by-Step Breakdown

Permissions on Files vs Directories

Permission	On a File	On a Directory
r	Read file content	List directory contents (ls)
w	Modify file content	Create/delete files inside
x	Execute the file	Enter the directory (cd)

You can delete a file you don't own if you have **write on the directory** — this surprises many candidates.

4. Key Commands & Examples

chmod — Change Permissions

bash

Symbolic mode

```
chmod u+x file      # Add execute for owner
chmod g-w file      # Remove write from group
chmod o=r file      # Set others to read only
chmod a+x file      # Add execute for all (a = all)
chmod u+x,g-w file  # Multiple changes
```

Octal mode (faster, used in production)

```
chmod 755 script.sh
chmod 644 config.conf
chmod 600 ~/.ssh/id_rsa
chmod 400 secret.pem
```

Recursive (use carefully)

```
chmod -R 755 /var/www/html
```

chown — Change Owner & Group

bash

Change owner

```
chown alice file.txt
```

Change owner and group

```
chown alice:developers file.txt
```

```
# Change group only
```

```
chown developers file.txt
```

```
# OR
```

```
chgrp developers file.txt
```

```
# Recursive
```

```
chown -R www-data:www-data /var/www/html
```

```
# Verify
```

```
ls -la file.txt
```

```
stat file.txt
```

Special Permission Bits — The Interview Goldmine

SUID (Set User ID) — Bit value: 4

```
bash
```

```
# When set on a binary:
```

```
# Executes as the FILE OWNER, not the calling user
```

```
# Classic example: passwd command
```

```
ls -l /usr/bin/passwd
```

```
# -rwsr-xr-x root root → 's' in owner execute = SUID set
```

```
# Any user runs passwd as root temporarily
```

```
# Set SUID
```

```
chmod u+s /path/to/binary
```

```
chmod 4755 /path/to/binary
```

```
# Find all SUID binaries on system (security audit)
```

```
find / -perm -4000 -type f 2>/dev/null
```

SGID (Set Group ID) — Bit value: 2

```
bash
```

```
# On a BINARY: executes as the file's GROUP
```

```
# On a DIRECTORY: new files inherit the directory's group
```

```
# (critical for shared team directories)
```

```
# Example: shared project directory
```

```
chmod g+s /shared/project
```

```
# Now all files created inside inherit 'project' group
```

```
# Set SGID
```

```
chmod g+s /dir
```

```
chmod 2755 /dir
```

```
# Find all SGID files
find / -perm -2000 -type f 2>/dev/null
```

Sticky Bit — Bit value: 1

```
bash
# On a DIRECTORY:
# Users can only delete files they OWN
# Even if they have write on the directory

# Classic example: /tmp
ls -ld /tmp
# drwxrwxrwt → 't' in others execute = sticky bit

# Set sticky bit
chmod +t /shared/dir
chmod 1777 /tmp

# Full special bits in octal:
# SUID + SGID + Sticky = 4+2+1 = 7
# chmod 7755 = SUID + SGID + Sticky + rwxr-xr-x
```

Special Bits — Quick Reference

Bit	Octal	On File	On Directory
SUID	4	Runs as file owner	No effect
SGID	2	Runs as file group	New files inherit group
Sticky	1	No effect	Only owner can delete their files

umask — Default Permission Filter

```
bash
# umask SUBTRACTS from default permissions
# Default for files: 666
# Default for dirs: 777

# If umask = 022:
# Files: 666 - 022 = 644 (rw-r--r--)
# Dirs: 777 - 022 = 755 (rwxr-xr-x)

# Check current umask
umask          # Shows: 0022

# Set umask for session
```

umask 027 # Files=640, Dirs=750 (more restrictive)

Permanent: add to ~/.bashrc or /etc/profile

ACLs — Fine-Grained Permissions

bash

Check if ACL is set ('+' sign in ls output)

ls -la file.txt

-rw-r--r--+ ← '+' means ACL exists

View ACL

getfacl file.txt

Set ACL for a specific user

setfacl -m u:bob:rw file.txt # Give bob read+write

setfacl -m u:alice:r file.txt # Give alice read only

setfacl -m g:devteam:rx /project # Group devteam gets rx

Set default ACL on directory (inherited by new files)

setfacl -d -m u:bob:rw /shared/dir

Remove specific ACL entry

setfacl -x u:bob file.txt

Remove ALL ACLs

setfacl -b file.txt

Recursive ACL

setfacl -R -m u:bob:rx /project

ACL Output — How to Read It

bash

getfacl /shared/project

file: shared/project

owner: alice

group: devs

user::rwx ← owner permissions

user:bob:rw- ← bob specifically gets rw

group::r-x ← group permissions

group:devteam:rwx ← devteam group gets rwx

mask::rwx ← maximum effective permissions

other::r-- ← everyone else

5. Interview Questions & Answers

Q1. What is SUID and why is it a security risk?

SUID on a binary means it runs with the **file owner's privileges** regardless of who executes it. `/usr/bin/passwd` uses SUID to let normal users update `/etc/shadow` (root-owned).

Security risk: if a **SUID root binary has a vulnerability**, an attacker can exploit it to **run arbitrary code as root**. This is a classic **privilege escalation vector**.

Always audit: `find / -perm -4000 -type f 2>/dev/null`

Q2. A user can't delete a file they own inside `/tmp`. Why?

Trick question — `/tmp` has the **sticky bit** set (`1777`). Sticky bit means you can **only delete files you own**, regardless of directory write permission. If the file is owned by someone else, you cannot delete it even with write on the directory.

Q3. Difference between `chmod 755` and `chmod 644` — when do you use each?

`755` → `rw-r-xr-x` — owner can read/write/execute, others can read and execute. Used for **executables, scripts, directories** that need to be traversed.

`644` → `rw-r--r--` — owner can read/write, others read only. Used for **config files, web files, static content** — readable but not executable.

Rule of thumb: **files = 644, directories/scripts = 755, private keys = 600**

Q4. What is the difference between standard permissions and ACLs?

Standard permissions support only **3 entities**: owner, group, others. If you need different permissions for **multiple users or groups** on the same file, you need ACLs.

Example: `/project` owned by `alice:dev`. You want `bob` to have read-only and `charlie` to have no access at all — impossible with standard permissions alone. ACL solves this with `setfacl`.

Q5. What does the `mask` in ACL output mean?

The **mask** defines the **maximum effective permissions** any named user or group ACL entry can have. Even if you grant **user:bob:rwX**, if **mask::r-x**, bob's effective permission is **r-x**.

The mask is automatically recalculated when you run **chmod** on a file with ACLs — this is a **common gotcha** in production.

6. Real-World Scenario

Scenario: A shared **/data/reports** directory is used by 3 teams — **finance**, **devops**, and **audit**. Requirements:

- **finance** → full read/write
- **devops** → read only
- **audit** → read only, cannot be changed even by accident
- New files must inherit these permissions automatically

Solution:

```
bash
# Create directory
mkdir /data/reports
chown root:finance /data/reports
chmod 770 /data/reports

# Set SGID so new files inherit group
chmod g+s /data/reports

# ACL for devops group
setfacl -m g:devops:rx /data/reports

# ACL for audit group
setfacl -m g:audit:rx /data/reports

# Default ACLs (inherited by new files)
setfacl -d -m g:finance:rwX /data/reports
setfacl -d -m g:devops:rx /data/reports
setfacl -d -m g:audit:rx /data/reports

# Verify
getfacl /data/reports
```

Topic 5: Memory Management

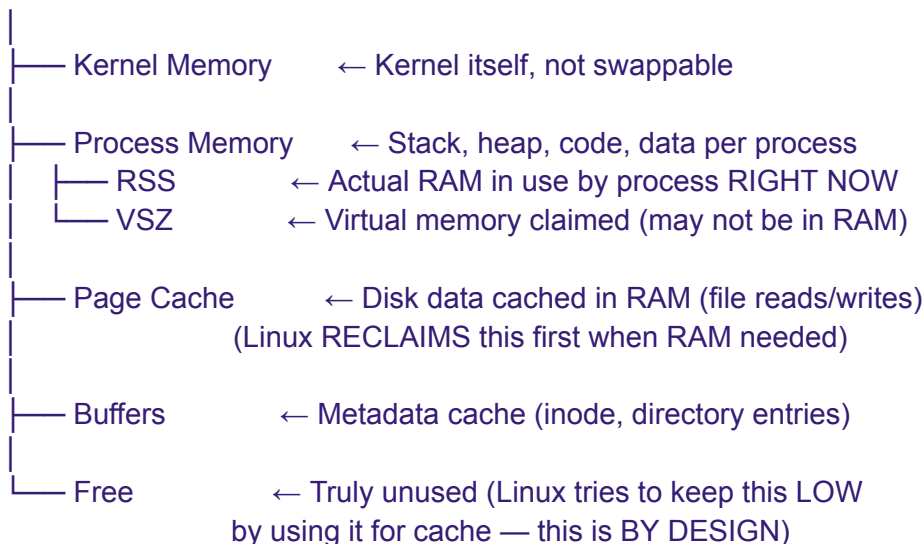
RAM, Swap, OOM Killer, Cache, /proc/meminfo

1. Core Concept

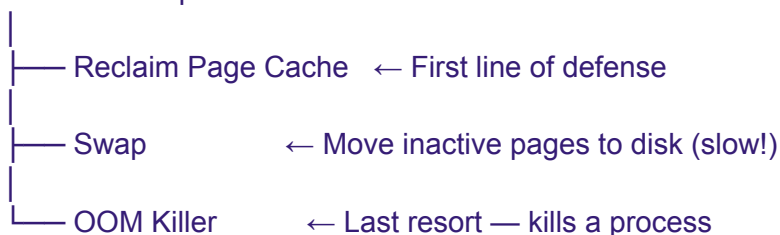
Linux memory management is **not what it looks like on the surface**. "Used" memory doesn't mean "wasted" — Linux aggressively uses free RAM for **cache/buffers** to speed up I/O. The OOM (Out-Of-Memory) Killer is Linux's last resort when RAM + swap are exhausted — it kills processes to survive. Misreading memory leads to **wrong capacity decisions in production**.

2. Step-by-Step Flow — How Linux Uses Memory

Physical RAM



When RAM fills up:



3. Interview Questions & Answers

Q1. Linux shows only 100MB free RAM. Should you be worried?

Not necessarily. Linux uses free RAM for **page cache** — caching disk data to speed up I/O. This cached memory is immediately reclaimable. Always check

the **available** column in `free -h`, not `free`. If **available** is low AND swap usage is growing, then there's a real problem.

Q2. What is the OOM Killer and how does it decide what to kill?

When the system runs out of RAM and swap, the kernel's OOM Killer selects a process to terminate to free memory. It calculates an **oom_score** per process based on: memory usage (largest consumer scores highest), runtime, and CPU usage. You can protect critical processes by setting `oom_score_adj` to `-1000` — the kernel will never kill them. Check OOM events with `dmesg | grep -i oom`.

Q3. What is the difference between RSS and VSZ?

VSZ is virtual memory — total address space the process has **mapped**, including memory that may not be loaded in RAM yet. **RSS** is resident set size — actual **physical RAM** the process is using right now. VSZ is always $\geq$ RSS. For capacity planning, use **RSS**. VSZ can be misleadingly large because of memory-mapped libraries and pre-allocated heaps.

Q4. Production server is swapping heavily. How do you troubleshoot?

bash

Step 1: Confirm swap usage

`free -h`

`vmstat 2 5` # Watch si/so columns

Step 2: Find what's in swap

`smem -r | head -10` # Or:

`for file in /proc/*/status; do`

`pid=$(echo $file | cut -d/ -f3)`

`name=$(cat /proc/$pid/comm 2>/dev/null)`

`vmswap=$(grep VmSwap $file 2>/dev/null | awk '{print $2}')`

`echo "$vmswap KB $name (PID $pid)"`

`done | sort -rn | head -10`

Step 3: Check swappiness

`cat /proc/sys/vm/swappiness`

Step 4: Fix

Reduce swappiness

`sysctl vm.swappiness=10`

Identify memory leak in top consumer
Add RAM or reduce application memory footprint

Q5. What are Huge Pages and when are they used?

By default Linux uses **4KB memory pages**. Huge Pages are **2MB or 1GB pages** — reduces the number of page table entries and TLB misses, improving performance for memory-intensive apps.

Used by: **Oracle DB, PostgreSQL shared_buffers, KVM/QEMU VMs, Java with `-XX:+UseHugePages`**.

Check: `cat /proc/meminfo | grep Huge` Configure: `sysctl vm.nr_hugepages=512`

4. Real-World Scenario

Scenario: App team says the Java app is running slow and memory is "full". You SSH in and need to diagnose.

bash

Step 1: Get the real picture

`free -h`

total=32G, used=30G, free=200M, available=8G

→ available=8G means page cache is reclaimable. Not in danger yet.

Step 2: Is swap being used?

`vmstat 2 5`

si=0, so=0 → no swap activity. System is fine.

Step 3: Find the Java process memory

`ps aux --sort=-%mem | grep java`

java PID RSS=22G ← This is the real consumer

Step 4: Check OOM risk

`cat /proc/<java_pid>/oom_score` # High? Protect it:

`echo -500 > /proc/<java_pid>/oom_score_adj`

Step 5: Check if it's growing (memory leak?)

`watch -n 5 'ps -o pid,rss,vsz -p <PID>'`

Step 6: Check GC logs (Java-specific)

JVM heap exhaustion ≠ OS memory issue

→ Tune -Xmx, check GC pressure, heap dumps

Conclusion: OS memory is healthy.
Problem is JVM heap tuning — escalate to app team with data.

Topic 6: systemd Deep Dive

Units, Targets, Dependencies, journald, Timers

1. Core Concept

systemd is PID 1 — the first process the kernel starts, and the parent of everything else. It replaces the old SysV init system with **parallel service startup, dependency management, and unified logging**. In production, every service you manage lives as a **unit file** — understanding this is non-negotiable for any DevOps/SRE role.

2. Step-by-Step Flow — How systemd Boots the System

Kernel starts /sbin/init → symlink to systemd (PID 1)



systemd reads default.target
(usually graphical.target or multi-user.target)



Resolves dependency chain (Wants, Requires, After, Before)



Activates units IN PARALLEL where possible



- mounts filesystems (*.mount units)
- brings up networking (network.target)
- starts system services (*.service units)
- mounts swap (*.swap units)
- starts timers (*.timer units)



Reaches target state → login prompt available

3. Interview Questions & Answers

Q1. What happens when you run `systemctl restart` vs `systemctl reload`?

restart = **stop + start** — kills the process (SIGTERM → SIGKILL) and starts a new one. All connections are dropped. New PID.

reload = sends **SIGHUP** (or ExecReload command) to the running process — process re-reads its config **without stopping**. Same PID, zero downtime. Only works if the application supports it (nginx, apache, sshd do). Always prefer **reload** in production.

Q2. You edited a unit file but **systemctl restart** shows old config. Why?

systemd **caches unit files in memory**. After any edit to a unit file, you **must run **systemctl daemon-reload**** first. Without it, systemd uses the cached version. This is one of the most common mistakes in production. Always: edit → **daemon-reload** → **restart**.

Q3. What is the difference between **Requires** and **Wants**?

Requires= is a **hard dependency** — if the required unit fails to start, this unit fails too. They're also stopped together.

Wants= is a **soft dependency** — systemd tries to start the wanted unit, but if it fails, this unit **continues anyway**.

Best practice: use **Wants**= + **After**= together. This starts units in the right order without creating brittle hard dependencies that cause cascading failures.

Q4. A service keeps crashing in production. How do you diagnose with systemd?

bash

Step 1: Check current status

systemctl status myapp # Shows last few log lines + exit code

Step 2: Full logs with context

journalctl -u myapp -n 100 # Last 100 lines

journalctl -u myapp --since "10 min ago"

Step 3: Check exit code

systemctl show myapp -p ExecMainStatus # Non-zero = crashed

Step 4: Check restart count

systemctl show myapp -p NRestarts

```
# Step 5: Look for OOM kill
journalctl -k | grep -i oom
```

```
# Step 6: Fix restart loop — temporarily disable restart
# Edit unit: Restart=no → daemon-reload → restart manually
```

Q5. What is socket activation in systemd?

systemd creates and listens on a socket **before** the service starts. When a connection arrives, systemd starts the service and hands the socket over. Benefits: **faster boot** (service starts only when needed), **no missed connections** (systemd buffers them), and services can be **restarted without dropping connections** (socket stays open).

Classic example: `sshd.socket` activates `sshd.service` on incoming connections.

4. Real-World Scenario

Scenario: A microservice deployed on a Linux server crashes intermittently under load. On-call gets paged at 3am. The service has restarted 47 times in the last hour.

```
bash
```

```
# Step 1: Assess the situation
systemctl status paymentservice
# → Active: activating (auto-restart) ...
# → NRestarts: 47
```

```
# Step 2: Get crash context
journalctl -u paymentservice -n 200 --since "1 hour ago"
# → Found: "FATAL: cannot connect to database: connection refused"
```

```
# Step 3: Check if it's OOM
journalctl -k | grep -i oom | tail -20
# → Clean — not OOM
```

```
# Step 4: Check DB service
systemctl status postgresql
# → FAILED — postgresql crashed due to disk full on /var
```

```
# Step 5: Fix root cause
df -h          # /var at 100%
du -sh /var/log/postgresql/* # 40GB log file
> /var/log/postgresql/pg.log # Truncate
systemctl start postgresql
```

Step 6: Service recovers automatically due to Restart=always

Step 7: Post-incident — add dependency to unit file

Requires=postgresql.service

After=postgresql.service

So service doesn't restart-loop when DB is down

systemctl daemon-reload

Topic 7: Linux Networking

ss, netstat, routing, iptables, DNS, troubleshooting

1. Core Concept

Linux networking is built on **layers** — interface → IP → routing → firewall → socket. Every packet travels through all of them. In interviews and production, you need to trace a network problem **systematically from layer 1 to layer 7**. Knowing **ss**, routing tables, **iptables**, and DNS resolution flow separates real engineers from tutorial followers.

2. Step-by-Step Flow — Packet Journey on Linux

Application (curl, nginx, app)



Socket (TCP/UDP port)



Kernel Network Stack



iptables / nftables ← Firewall rules evaluated here



Routing Table ← Which interface/gateway to use?



Network Interface (eth0, ens3, bond0)



Physical / Virtual NIC



3. Interview Questions & Answers

Q1. What is the difference between `ss` and `netstat`?

`netstat` is part of the deprecated `net-tools` package — slow on systems with many connections because it reads from `/proc/net/tcp`. `ss` uses the **netlink socket interface** directly — faster, more detailed, and actively maintained. Both show sockets, but `ss` is the modern standard. In interviews, use `ss -tulnp` and mention that `netstat` is legacy.

Q2. You see thousands of `TIME_WAIT` connections. What does it mean?

`TIME_WAIT` is normal — after a TCP connection closes, the **initiating side waits $2 \times \text{MSL}$ (Maximum Segment Lifetime, ~60s)** to ensure the final ACK was received. Under high traffic (HTTP APIs, load balancers), thousands are expected.

Problem only if: ports are exhausted (`ss -s` shows near 60,000 connections).
Fix:

```
bash
sysctl -w net.ipv4.tcp_tw_reuse=1    # Reuse TIME_WAIT sockets
sysctl -w net.ipv4.ip_local_port_range="1024 65535" # Expand port range
```

Q3. `CLOSE_WAIT` connections are accumulating. What's the cause?

`CLOSE_WAIT` means the **remote side closed the connection but the local application hasn't closed its socket yet**. This is always an **application bug** — the app is not properly handling connection close events. The fix is in the application code, not the OS. Accumulating `CLOSE_WAIT` = connection/file descriptor leak → eventually app crashes with "too many open files."

Q4. How does a DNS query resolve `google.com` step by step?

1. Check `/etc/hosts` — any entry? If yes, done.
2. Check local DNS cache (systemd-resolved / nsd)
3. Query DNS server in `/etc/resolv.conf`
4. DNS server checks its cache — if miss:
5. Queries **root nameservers** (.)

6. Root refers to **TLD nameservers** (.com)
 7. TLD refers to **authoritative nameservers** for google.com
 8. Authoritative NS returns the A record (IP)
 9. Response cached per TTL, returned to application
-

Q5. App can't connect to DB on port 5432. How do you troubleshoot?

bash

Step 1: Is DB listening?

ss -tlnp | grep 5432 # On DB server

Step 2: Can we reach the port?

nc -zv db-server 5432 # From app server

Step 3: Firewall blocking?

iptables -L -n -v | grep 5432 # On DB server

Check security groups if cloud (AWS/GCP)

Step 4: Is DB bound to right interface?

ss -tlnp | grep 5432

127.0.0.1:5432 = only localhost — not accessible remotely!

0.0.0.0:5432 = all interfaces — accessible

Step 5: Routing issue?

ip route get <db-server-ip> # From app server

Step 6: DNS resolution?

dig db-server # Does name resolve correctly?

cat /etc/hosts | grep db # Any override?

Step 7: Packet capture (definitive)

tcpdump -i eth0 port 5432 # On DB server — do packets arrive?

4. Real-World Scenario

Scenario: Production API server stops accepting new connections. Existing connections work. Users get "connection refused" errors.

bash

Step 1: Check if service is listening

ss -tlnp | grep :8080

→ port 8080 not in list → service crashed or wrong port

Step 2: Is the process running?

systemctl status api-service

→ Active: failed

Step 3: Check logs

journalctl -u api-service -n 50

→ "bind: address already in use"

Step 4: Find what's using port 8080

ss -tlnp | grep :8080 # Nothing listening now

lsof -i :8080 # Any process holding it?

Step 5: Check for zombie/stuck processes

ss -tnp | grep 8080 # CLOSE_WAIT connections still holding port?

Step 6: Check file descriptor limit (common under load)

ulimit -n # Current FD limit

cat /proc/\$(pgrep api)/limits # Process-level limits

If FD limit hit → "too many open files" in logs

Fix FD limits in systemd unit:

[Service]

LimitNOFILE=65536

Step 7: Restart and monitor

systemctl start api-service

watch -n 1 'ss -s' # Monitor socket counts live

Topic 8: Disk & LVM Management

Partitions, LVM, Resize, fstab, fsck

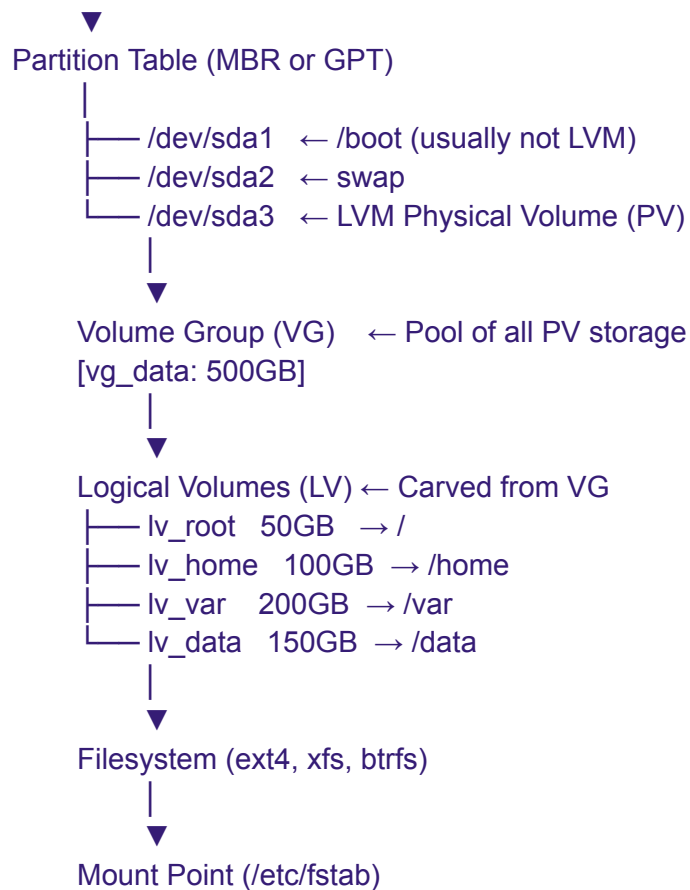
1. Core Concept

Linux disk management has **two layers** — physical partitioning (MBR/GPT) and logical volume management (LVM). LVM sits **on top of** physical partitions and gives you superpowers: **resize volumes online, add disks, snapshot, stripe** — without downtime. In production, almost every enterprise Linux server uses LVM. **fstab** is what makes mounts survive reboots — one wrong entry and the server won't boot.

2. Step-by-Step Flow — Full Disk Stack

Physical Disk (/dev/sda, /dev/nvme0n1)

|



3. Interview Questions & Answers

Q1. Walk me through extending a logical volume online in production.

```
bash
# 1. Confirm VG has free space
vgs                # Check VFree column

# 2. Extend LV + filesystem in one command
lvextend -L +20G -r /dev/vg_data/lv_var
# -r flag resizes filesystem automatically (ext4 and xfs)

# 3. Verify
df -h /var
ls
```

Key point: **no downtime, no unmount needed** for extending. This is the main advantage of LVM over raw partitions.

Q2. What happens if you shrink an LV before shrinking the filesystem?

Data loss/corruption. The LV tells the filesystem it has less space than the filesystem expects — the filesystem's data structures at the end of the volume are now outside the LV boundary. The filesystem becomes corrupted. Always: **shrink filesystem first, then shrink LV**. And always backup before any shrink operation. Also remember: **XFS cannot be shrunk at all** — only grown.

Q3. What is the difference between **ext4** and **xfs**?

	ext4	xfs
Default on	Debian/Ubuntu	RHEL/CentOS 7+
Max file size	16TB	8EB
Max FS size	1EB	8EB
Can shrink?	Yes	No
Online grow?	Yes	Yes
Best for	General use	Large files, high throughput
Journal	Yes	Yes (better performance)

In interviews: **xfs is preferred for large-scale data workloads** (databases, media). **ext4 is the safe general-purpose choice**.

Q4. Server dropped to emergency shell after reboot — what happened?

99% chance: **bad /etc/fstab entry**. A UUID that doesn't exist, wrong filesystem type, or missing device causes the kernel to fail mounting a filesystem listed in fstab.

Fix from emergency shell:

```
bash
mount -o remount,rw /      # Remount root as read-write
vim /etc/fstab             # Fix the bad entry
mount -a                  # Test — should show no errors
reboot
```

Prevention: **always run `mount -a` after editing fstab**.

Q5. What is an LVM snapshot and how is it used in production?

A snapshot is a **point-in-time copy** of an LV. It uses **copy-on-write** — only stores blocks that change after the snapshot, so initial space usage is small. Production uses:

1. **Pre-patching snapshots** — snapshot before OS updates, merge to rollback if broken
2. **Consistent backups** — snapshot running DB, backup the snapshot (avoids backing up changing data)
3. **Testing** — snapshot prod-like data, mount it in test environment

Caveat: **snapshots degrade LV performance** as they fill up — remove promptly after use.

4. Real-World Scenario

Scenario: 3am alert — `/var` at 100% on a production database server. Can't add a new disk right now, but another LV (`/data`) has 200GB free space in the same VG.

bash

Step 1: Confirm the problem

`df -h`

`/var` is 100%, `/data` has 150GB used of 350GB

Step 2: Check VG free space

`vgs`

`VFree = 0` — no free space in VG directly

Step 3: Shrink `/data` LV to free up VG space

(ext4 only — confirm type first)

`df -hT /data` # ext4 confirmed

Shrink `/data` from 350GB to 200GB (free 150GB)

`umount /data` # Must unmount for shrink

`e2fsck -f /dev/vg_data/lv_data` # Check FS first

`resize2fs /dev/vg_data/lv_data 180G` # Shrink FS (smaller than LV target)

`lvreduce -L 200G /dev/vg_data/lv_data` # Shrink LV

`mount /data` # Remount

Step 4: Extend `/var` with freed space

`lvextend -L +100G -r /dev/vg_data/lv_var`

Step 5: Verify

`df -h /var` # Now has headroom

`df -h /data` # Still functional

```
# Step 6: Root cause — what filled /var?  
du -sh /var/log/*  
# → PostgreSQL logs not rotating  
# Fix logrotate, set up monitoring alert at 80%
```

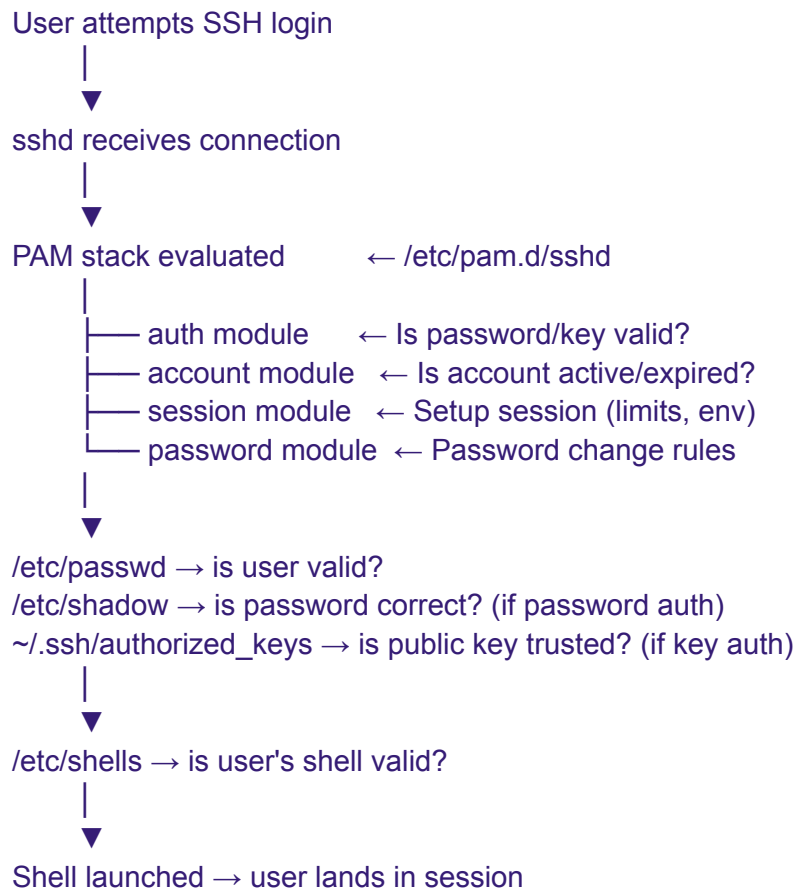
Topic 9: Users, Groups & SSH

passwd, shadow, sudo, PAM, SSH key auth, hardening

1. Core Concept

Linux access control starts with **users and groups** — every process, file, and resource is owned by one. **SSH** is the primary entry point into every production Linux server — misconfiguring it is a security incident waiting to happen. **PAM** (Pluggable Authentication Modules) is the invisible layer that controls HOW authentication works — understanding it separates seniors from juniors.

2. Step-by-Step Flow — Login Authentication Chain



3. Interview Questions & Answers

Q1. What is the difference between **su** and **sudo**?

su (switch user) **fully switches to another user** — requires that user's password, gives a complete new shell as that user. **su** - loads the full environment.

sudo runs a **single command** with elevated privileges — requires YOUR OWN password, logs every command to `/var/log/auth.log` or `journalctl`, and respects `/etc/sudoers` rules.

Production best practice: **disable root login, use sudo exclusively** — every privileged action is auditable by username.

Q2. User can't login via SSH with their key — how do you troubleshoot?

bash

Step 1: Test with verbose output (on client)

`ssh -vvv alice@server` # Shows exact failure point

Step 2: Check sshd config allows key auth

`grep -i pubkey /etc/ssh/sshd_config`

PubkeyAuthentication yes

Step 3: Check permissions (SSH is STRICT about this)

`ls -la ~alice/.ssh/`

.ssh must be 700

authorized_keys must be 600

owned by alice, not root

Step 4: Check SELinux context (RHEL)

`ls -Z ~alice/.ssh/authorized_keys`

`restorecon -Rv ~alice/.ssh/`

Step 5: Check auth log

`tail -f /var/log/auth.log` # Ubuntu

`journalctl -u sshd -f` # systemd

Step 6: Is the public key actually in authorized_keys?

`cat ~alice/.ssh/authorized_keys`

Step 7: Check if account is locked

`passwd -S alice` # Shows account status

Q3. What is PAM and why does it matter?

PAM (Pluggable Authentication Modules) is a **framework that decouples authentication from applications**. Instead of each app implementing its own auth, they call PAM, and PAM's configuration determines HOW authentication happens.

Why it matters: you can enforce **MFA, account lockout, session limits, LDAP auth** across ALL services (SSH, sudo, login) by changing PAM config — without touching any application. It's the central authentication control plane of Linux.

Q4. What happens when you run `sudo` command?

1. `sudo` reads `/etc/sudoers` (via `sudoers` PAM) to check if user has permission
 2. PAM stack in `/etc/pam.d/sudo` validates the user's password
 3. If authorized, `sudo` forks a child process with target user's privileges (default: root)
 4. Command runs as root, output returned to terminal
 5. Every sudo invocation is **logged** to `/var/log/auth.log` / `journalctl` with: timestamp, user, terminal, command
 6. `sudo` caches credentials for 5 minutes (configurable with `timestamp_timeout`)
-

Q5. What is the danger of `usermod -G` vs `usermod -aG`?

`usermod -G docker alice` **replaces** alice's entire supplementary group list with just `docker`. If alice was previously in `wheel`, `developers`, `sshusers` — she loses all those memberships instantly.

`usermod -aG docker alice` **appends** docker to her existing groups.

This is a **production incident** waiting to happen — removing someone from `wheel` accidentally can lock out sudo access. **Always use `-aG`**. Verify after with `id alice`.

4. Real-World Scenario

Scenario: Security audit requires: disable password SSH auth, enforce key-only login, lock out accounts after 3 failed attempts, and ensure all privileged actions are logged.

```
bash
# Step 1: Ensure all admins have keys loaded BEFORE disabling passwords
# (Don't lock yourself out)
ssh-copy-id -i ~/.ssh/id_ed25519.pub alice@server

# Step 2: Harden sshd_config
vim /etc/ssh/sshd_config
# Set:
PasswordAuthentication no
PermitRootLogin no
MaxAuthTries 3
AllowGroups sshusers

# Step 3: Test config syntax (CRITICAL before reload)
sshd -t          # Must show no errors

# Step 4: Add admins to sshusers group
groupadd sshusers
usermod -aG sshusers alice
usermod -aG sshusers bob

# Step 5: Reload SSH (not restart — keeps existing sessions alive)
systemctl reload sshd

# Step 6: Configure account lockout via PAM
vim /etc/pam.d/sshd
# Add faillock configuration

# Step 7: Configure sudo logging
vim /etc/sudoers
# Defaults logfile=/var/log/sudo.log
# Defaults log_input, log_output

# Step 8: Verify from another terminal (DON'T close current session yet)
ssh -i ~/.ssh/id_ed25519 alice@server # Test key auth works
ssh alice@server                      # Test password fails

# Step 9: Monitor
tail -f /var/log/auth.log
journalctl -u sshd -f
```

Topic 10: Linux Performance Troubleshooting

Full SRE Playbook — CPU, Memory, Disk, Network

1. Core Concept

Performance troubleshooting is **systematic elimination**, not random guessing. Every production issue maps to one or more of **four resource bottlenecks**: CPU, Memory, Disk I/O, Network. The SRE approach: **observe** → **measure** → **isolate** → **fix** → **verify**. Interviewers give you a scenario — they want to see your **thought process**, not just commands.

2. The Golden Framework — USE Method

For every resource, ask THREE questions:

U → Utilization Is the resource being used near capacity?

S → Saturation Is work queuing up waiting for the resource?

E → Errors Are there errors on this resource?

Apply to:

└─ CPU → %util, load average, run queue, errors

└─ Memory → % used, swap activity, OOM events

└─ Disk → %util, I/O wait, queue depth, errors

└─ Network → bandwidth %, drops, retransmits, errors

This framework = your interview answer structure.

Never jump to commands. State the framework first.

3. Interview Questions & Answers

Q1. Server load average is 40 on an 8-core machine. Walk me through your investigation.

Bash

Load 40 on 8-core = 5x saturation. Something is very wrong.

Step 1: What's in the run queue?

`top` # How many processes in R state?

`vmstat 1 3` # r column = run queue depth

Step 2: Is it CPU or I/O?

`top` # Check wa% (iowait)

High wa% → disk/NFS bottleneck causing load (not CPU)

Low wa%, high us% → actual CPU bottleneck

Step 3: Identify top consumers

`ps aux --sort=-%cpu | head -10`

or

`ps aux | awk '{print $8, $0}' | grep "^R" | head -20` # Running processes

Step 4: Are there D-state processes? (uninterruptible I/O wait)

`ps aux | grep " D "` # D state = waiting on I/O

`dmesg | grep -i "nfs\|hung\|timeout"` # NFS or storage issue?

Step 5: Check I/O

`iostat -xz 1` # %util 100% on any disk?

`df -h` # Any filesystem full?

Most common cause of load 40: hung NFS mount

Every process trying to access it goes D state

Fix: `umount -f -l /hung/mount` or restart nfs service

Q2. Application response time suddenly doubled. No alerts firing. What do you check?

bash

No single resource at 100% = subtle bottleneck

Step 1: Establish baseline comparison

"Suddenly doubled" — when did it start?

sar -u -r -d -n DEV # Historical stats from sar

Step 2: Check all four resources

uptime # Load average trend

free -h # Memory pressure?

iostat -xz 1 # Disk await time increase?

ss -s # Connection count spike?

Step 3: Application-level

Check app logs for slow queries, timeouts, errors

journalctl -u myapp --since "1 hour ago" | grep -i "slow|timeout|error"

Step 4: External dependencies

time dig internal-db-server # DNS slow?

nc -zv db-server 5432 # DB port responsive?

time curl -o /dev/null http://api.internal # Internal API slow?

Step 5: Network

netstat -s | grep retransmit # Packet loss?

ping -c 100 db-server | tail # Packet loss to DB?

Step 6: Check for noisy neighbor (VM/container environment)

Other VMs stealing CPU/IO from hypervisor

top → check steal time (%st) # st > 5% = hypervisor stealing CPU

Fix: escalate to infra team, live migrate VM

Q3. Disk I/O wait is high. How do you find what's causing it?

bash

Step 1: Confirm and measure

iostat -xz 1

await > 50ms on HDD or > 5ms on SSD = problem

%util approaching 100% = disk saturated

Step 2: Find culprit process

iotop -o # Show only active I/O processes

pidstat -d 1 # I/O per process

Step 3: What files?

lsuf -p <culprit_PID> # What files is it accessing?

`strace -e trace=read,write -p <PID> # What exactly is it doing?`

Step 4: What type of I/O?

`iostat` → check r/s vs w/s # Mostly reads or writes?

High writes → is it logging? flush heavy? database writes?

High reads → cache cold? reading large dataset repeatedly?

Step 5: Is it a specific filesystem/mount?

`iostat -xz 1` # Which device: sda? sdb? dm-0?

`lsblk` # Map dm-0 to LVM → to mount point

`df -h` # Which mount point?

Step 6: Fix options

Immediate: `ionice -c 3 -p <PID>` (lower I/O priority)

Short-term: move hot data to faster disk (SSD)

Long-term: add caching layer (Redis), optimize queries, tune I/O scheduler

Q4. Memory is fine, CPU is fine, disk is fine — but app is slow. What now?

`bash`

When obvious resources are fine → look at:

1. Network latency to dependencies

`for host in db-server cache-server api-server; do`

`echo -n "$host: "`

`ping -c 5 $host | tail -1`

`done`

2. Connection pool exhaustion

`ss -tnp | grep ESTABLISHED | wc -l` # Total connections

`ss -tnp | grep <app_port> | wc -l` # App-specific

3. Too many TIME_WAIT / CLOSE_WAIT

`ss -s` # State breakdown

CLOSE_WAIT accumulating = app not closing connections

4. File descriptor exhaustion

`lsof -p <PID> | wc -l` # FDs open by process

`cat /proc/<PID>/limits` # FD limit for process

If near limit → "too many open files" errors incoming

5. DNS resolution latency (often missed)

`time getent hosts db-server` # How long does internal DNS take?

> 50ms = DNS problem

6. Garbage collection (Java/Go/Python)

Java: `jstat -gcutil <PID> 1000` # GC frequency/duration

High GC pause → app freezes periodically

```
# 7. Lock contention
# Database: check slow query log, lock waits
# App: thread dumps (Java: kill -3 <PID>)

# 8. Context switches (too many threads fighting for CPU)
vmstat 1          # cs column = context switches/sec
pidstat -w -p <PID> 1    # Per-process context switches
# Very high cs = too many threads, lock contention
```

Q5. How do you tell if a performance issue is CPU-bound vs I/O-bound?

```
bash
# CPU-bound:
# → %us or %sy high in top
# → wa% LOW (CPU not waiting for I/O)
# → load average high, iostat %util low
# → Processes in R state (running)
# Fix: optimize code, scale horizontally, add CPUs

# I/O-bound:
# → wa% HIGH in top (CPU sitting idle waiting for disk)
# → iostat %util high, await high
# → load average high, but CPU us% low
# → Processes in D state (uninterruptible sleep)
# Fix: faster storage, I/O caching, read replicas, query optimization

# Quick test:
vmstat 1 5
# r=run queue (CPU bound) vs b=blocked (I/O bound)
# b > 0 consistently = I/O bound problem

# The key insight:
# High load + high wa% = I/O problem (not CPU)
# High load + low wa% + high us% = CPU problem
# This distinction is asked in EVERY SRE interview
```

9. Real-World Scenario

Scenario: Monday 9am. Entire engineering team starts work. Production app degrades — p99 latency goes from 50ms to 8 seconds. On-call gets paged.

```
bash
# T+0: Acknowledge alert, open server terminal

# T+1min: 5-minute triage
uptime
# load average: 24.5, 18.2, 8.1 ← spike right now, was fine
```

```

free -h
# available: 2.1GB ← memory OK

iostat -xz 1
# sda: await=280ms %util=99% ← DISK IS SATURATED

ss -s
# ESTABLISHED: 4821 ← high but not alarming

# T+2min: Confirm disk is the bottleneck
iotop -o
# PID 8823 java   DISK READ: 450M/s ← THIS IS THE PROBLEM

# T+3min: What is this Java process?
ps -f -p 8823
# → Elasticsearch reindex job started at 9:01am

# T+4min: Impact assessment
# Elasticsearch reindex reading 450MB/s from disk
# Disk saturated → all other app queries waiting on disk
# await 280ms vs normal <5ms = 56x slower

# T+5min: Immediate mitigation
# Reduce I/O priority of reindex job
ionice -c 3 -p 8823      # Idle class — only uses I/O when free
renice -n 19 -p 8823     # Lowest CPU priority too

# T+7min: Verify improvement
iostat -xz 1
# await dropping: 280ms → 95ms → 40ms → 8ms

# T+10min: App latency recovering
# p99 dropping: 8s → 2s → 400ms → 65ms

# T+15min: Communicate resolution, document timeline

# Post-incident actions:
# 1. Schedule reindex jobs during off-hours (2am-5am)
# 2. Add cgroup I/O limits to reindex job
# 3. Set up disk await monitoring alert (threshold: >50ms)
# 4. Separate Elasticsearch data to dedicated disk
# 5. Add runbook entry for this pattern

```

Topic 11: Shell Scripting for DevOps

Bash Essentials, Patterns, Interview Scripts

1. Core Concept

Shell scripting is **glue code for infrastructure** — automating repetitive tasks, parsing logs, health checks, deployments, and alerting. In interviews, you won't be asked to write perfect code — you'll be asked to **think out loud, handle edge cases, and write production-safe scripts**. The difference between a junior and senior script: **error handling, input validation, and idempotency**.

2. Interview Questions & Answers

Q1. What does `set -euo pipefail` do and why is it important?

`-e` exits the script immediately when any command returns non-zero. `-u` treats unset variables as errors — catches typos like `$DIIR` silently evaluating to empty. `-o pipefail` makes a pipeline fail if ANY command in it fails — without it, `cat missing_file | grep foo` returns 0 because `grep` succeeded. Without these flags, scripts silently continue through errors producing unexpected results. In production, silent failures are more dangerous than loud crashes.

Q2. What is the difference between `$@` and `$*`?

Both represent all positional arguments, but differ when quoted:

- `"$@"` → each argument as a **separate quoted string**: `"arg1" "arg2" "arg3"` — **always use this**
- `"$*"` → all arguments as **one string**: `"arg1 arg2 arg3"` — breaks args with spaces

bash

Example: arg with spaces

script.sh "hello world" "foo"

for arg in "\$@"; do echo "\$arg"; done # Correct: 2 iterations

for arg in "\$*"; do echo "\$arg"; done # Wrong: 1 iteration

Q3. How do you handle a script that must not run twice simultaneously?

bash

Use a lock file with flock

LOCKFILE="/tmp/myscript.lock"

exec 200>"\$LOCKFILE"

flock -n 200 || die "Script already running (lock: \$LOCKFILE)"

OR simple PID lock

PIDFILE="/tmp/myscript.pid"

if [[-f "\$PIDFILE"]]; then

pid=\$(cat "\$PIDFILE")

```

if kill -0 "$pid" 2>/dev/null; then
    die "Already running with PID $pid"
fi
fi
echo $$ > "$PIDFILE"
trap 'rm -f "$PIDFILE"' EXIT

```

Q4. How do you make a script idempotent?

An idempotent script produces the **same result whether run once or ten times** — critical for deployment and configuration scripts.

Techniques:

```

bash
# Check before act
id alice &>/dev/null || useradd alice # Only create if not exists
[[ -d /opt/app ]] || mkdir -p /opt/app # Only mkdir if not exists

# Use tools that are idempotent by design
# cp overwrites = idempotent
# systemctl enable = idempotent (won't fail if already enabled)
# sed -i 's/old/new/' = idempotent if pattern not present

# Guard with grep before adding to files
grep -qF "vm.swappiness=10" /etc/sysctl.conf \
    || echo "vm.swappiness=10" >> /etc/sysctl.conf

```

Q5. Write a script to check if a service is up, restart it if not, and alert if restart fails.

```

bash
#!/usr/bin/env bash
set -euo pipefail

SERVICE="${1:?Usage: $0 <service_name>}"
ALERT_EMAIL="oncall@company.com"
MAX_RESTARTS=3
RESTART_COUNT=0

send_alert() {
    local subject="$1"
    local body="$2"
    echo "$body" | mail -s "$subject" "$ALERT_EMAIL" 2>/dev/null || \
        echo "[ALERT] $subject: $body" # Fallback to stdout
}

if systemctl is-active --quiet "$SERVICE"; then
    echo "$SERVICE is healthy"
    exit 0
fi

```

```

echo "$SERVICE is DOWN — attempting restart..."

while [[ $RESTART_COUNT -lt $MAX_RESTARTS ]]; do
    RESTART_COUNT=$((RESTART_COUNT + 1))
    echo "Restart attempt $RESTART_COUNT/$MAX_RESTARTS..."

    if systemctl restart "$SERVICE"; then
        sleep 5
        if systemctl is-active --quiet "$SERVICE"; then
            send_alert \
                "RESOLVED: $SERVICE restarted successfully" \
                "$SERVICE was down but recovered after $RESTART_COUNT restart(s) on"
            $(hostname)
            echo "$SERVICE recovered"
            exit 0
        fi
    fi
    sleep $((RESTART_COUNT * 10)) # Progressive backoff
done

send_alert \
    "CRITICAL: $SERVICE is DOWN on $(hostname)" \
    "$SERVICE failed to restart after $MAX_RESTARTS attempts. Manual intervention"
    required."
exit 2

```

3. Real-World Scenario

Scenario: You need a script that runs every 5 minutes via cron, checks if disk usage on any partition exceeds 85%, and sends a Slack alert with details — but only once per incident (no alert spam).

```

bash
#!/usr/bin/env bash
set -euo pipefail

THRESHOLD=85
SLACK_WEBHOOK="${SLACK_WEBHOOK:?'SLACK_WEBHOOK not set'}"
ALERT_DIR="/tmp/disk_alerts"
mkdir -p "$ALERT_DIR"

send_slack() {
    local message="$1"
    curl -s -X POST "$SLACK_WEBHOOK" \
        -H 'Content-type: application/json' \
        --data '{"text": \"$message\"}' > /dev/null
}

```

```
df -h --output=target,pcent | tail -n +2 | while read -r mount usage; do
    pct="${usage//%/}"          # Strip % sign
    [[ "$pct" =~ ^[0-9]+$ ]] || continue # Skip non-numeric

    alert_file="$ALERT_DIR/$(echo "$mount" | tr '/' '_').alert"

    if [[ $pct -ge $THRESHOLD ]]; then
        if [[ ! -f "$alert_file" ]]; then
            # First time — send alert and create sentinel file
            msg=":warning: *Disk Alert* on \`${hostname}\`"
            msg+="Mount: \`${mount}\` Usage: *${usage}* (threshold: ${THRESHOLD}%)"
            send_slack "$msg"
            touch "$alert_file"
        fi
        # Else: alert file exists = already notified, skip
    else
        # Usage back to normal — remove sentinel, send recovery
        if [[ -f "$alert_file" ]]; then
            send_slack ":white_check_mark: *Disk Recovered* \`${hostname}\` \`${mount}\`
is now at ${usage}"
            rm -f "$alert_file"
        fi
    fi
done
```

Topic 12: Linux Security & Hardening

SELinux, auditd, sysctl, SSH, CIS Benchmarks

1. Core Concept

Linux security is **layered defense** — no single tool protects everything. The model: **least privilege + audit everything + reduce attack surface**. In interviews, security questions test whether you understand **why** controls exist, not just how to configure them. In production, a misconfigured security control can lock out your entire team — understanding before touching is non-negotiable.

2. The Linux Security Layers

Layer 1: Physical / Boot Security

- └─ BIOS/UEFI password
- └─ Secure Boot
- └─ GRUB password

Layer 2: Authentication & Access

- └─ SSH hardening (key-only, no root)
- └─ PAM (lockout, MFA, complexity)

- └─ sudo (least privilege)
- └─ Password policies (chage)

Layer 3: Filesystem & Permissions

- └─ File permissions (chmod/chown)
- └─ ACLs (setfacl)
- └─ SUID/SGID audit
- └─ Immutable files (chattr +i)

Layer 4: Mandatory Access Control

- └─ SELinux (RHEL/CentOS) ← label-based MAC
- └─ AppArmor (Ubuntu/Debian) ← path-based MAC

Layer 5: Network Security

- └─ iptables / nftables (firewall)
- └─ sysctl network hardening
- └─ TCP wrappers (/etc/hosts.allow/deny)

Layer 6: Audit & Monitoring

- └─ auditd (kernel audit framework)
- └─ journald / syslog
- └─ AIDE / Tripwire (file integrity)
- └─ fail2ban (brute force protection)

Layer 7: System Hardening

- └─ sysctl (kernel parameters)
 - └─ ulimits (resource limits)
 - └─ CIS Benchmarks
 - └─ Unnecessary service removal
-

3. Interview Questions & Answers

Q1. What is the difference between DAC and MAC?

DAC (Discretionary Access Control) — the file owner controls permissions. Standard Linux `chmod/chown`. The weakness: root bypasses everything, and a compromised process running as root has unlimited access.

MAC (Mandatory Access Control) — the **kernel enforces a policy** that no user or process can override, including root. SELinux labels every process and file — a compromised nginx process labeled `httpd_t` cannot read `/etc/shadow` (labeled `shadow_t`) **even if nginx somehow gains root**. The kernel blocks it at the policy level. MAC is defense-in-depth against privilege escalation.

Q2. Web app deployed but getting 403 errors on RHEL. Files have correct permissions. What do you check?

```
bash
# Classic SELinux scenario on RHEL/CentOS

# Step 1: Is SELinux enforcing?
getenforce          # If Enforcing → suspect SELinux

# Step 2: Check for denials
ausearch -m avc -ts recent -i
# Look for: denied { read } for comm="nginx"

# Step 3: Temporarily verify it's SELinux
setenforce 0        # Permissive
# Test app — if 403 goes away → confirmed SELinux

# Step 4: Fix file context
ls -Z /var/www/html/ # What context do files have?
# If context is wrong (e.g. user_home_t instead of httpd_sys_content_t)
restorecon -Rv /var/www/html/

# Step 5: If custom directory
semanage fcontext -a -t httpd_sys_content_t '/custom/webroot(/.*)?'
restorecon -Rv /custom/webroot/

# Step 6: Re-enable
setenforce 1

# Common mistake: copying files from home directory
# Files inherit home_t context → nginx can't read them
# Fix: restorecon or chcon
```

Q3. How does auditd differ from regular system logging?

Regular logging (`syslog`, `journald`) captures **application-level events** — what apps choose to log. `auditd` operates at the **kernel system call level** — it captures events **before** the application can interfere or suppress them. Key differences: `auditd` can track **any file access, any syscall, any privilege change** regardless of whether the app logs it. It captures the real UID (AUID — audit UID) even after `su/sudo`. It cannot be disabled by a compromised root process (kernel-enforced). It's the difference between "what the app said happened" vs "what the kernel actually saw happen."

Q4. What `sysctl` settings would you apply to harden a production web server?

The critical ones:

- `net.ipv4.tcp_syncookies=1` — SYN flood (DDoS) protection
- `net.ipv4.conf.all.rp_filter=1` — anti-spoofing

- `net.ipv4.conf.all.accept_redirects=0` — prevent MITM via ICMP redirect
- `kernel.randomize_va_space=2` — full ASLR (memory exploit protection)
- `kernel.yama.pttrace_scope=1` — restrict process injection
- `fs.protected_symlinks=1` — prevent symlink attacks in /tmp
- `kernel.dmesg_restrict=1` — hide kernel info from non-root

Apply via `/etc/sysctl.d/99-hardening.conf` and `sysctl --system`

Q5. Production server was breached. How do you use auditd to investigate?

bash

Incident response with auditd

Step 1: When did the breach happen?

`ausearch -m USER_LOGIN -ts today -i` # Login events

`ausearch -m USER_LOGIN --start 01/01/2024 -i` # Specific date

Step 2: What commands did the attacker run?

`ausearch -m EXECVE -ts today -i | grep -A5 "auid=attacker_uid"`

Step 3: What files did they access/modify?

`ausearch -m OPEN -ts today -i | grep "name="`

`ausearch -k identity -ts today -i` # Identity file changes

Step 4: Did they escalate privileges?

`ausearch -m USER_CMD -ts today -i` # sudo commands

`ausearch -k priv_esc -ts today -i` # SUID executions

`ausearch -k suid_exec -ts today -i`

Step 5: Did they install anything?

`ausearch -k modules -ts today -i` # Kernel module loads

`ausearch -m SOFTWARE_UPDATE -ts today -i`

Step 6: What network connections?

auditd doesn't capture network directly — check:

`ss -tnp` # Current connections

`last` # Login history

`lastlog` # Last login per user

Step 7: Full timeline reconstruction

`ausearch -ts "01/15/2024 00:00:00" \`

`-te "01/15/2024 23:59:59" \`

`-i | aureport --summary`

4. Real-World Scenario

Scenario: Security team mandates: new production servers must pass CIS Level 1 benchmark, all privileged commands must be audited, and SSH must use keys only with brute force protection.

bash

— 1. SSH Hardening —————

```
cat >> /etc/ssh/sshd_config << 'EOF'
```

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
ClientAliveInterval 300
ClientAliveCountMax 2
AllowGroups sshusers
X11Forwarding no
EOF
```

```
sshd -t && systemctl reload sshd
```

— 2. PAM Lockout —————

```
# /etc/pam.d/system-auth — add faillock
```

```
authconfig --enablefaillock \
    --faillockargs="deny=3 unlock_time=600" \
    --update
```

— 3. Audit Rules —————

```
cat > /etc/audit/rules.d/cis.rules << 'EOF'
```

```
-D
-b 8192
-f 1
-w /etc/passwd -p wa -k identity
-w /etc/shadow -p wa -k identity
-w /etc/sudoers -p wa -k identity
-w /usr/bin/sudo -p x -k priv_esc
-a always,exit -F arch=b64 -S execve -k exec
-e 2
EOF
```

```
service auditd restart
```

— 4. sysctl Hardening —————

```
cat > /etc/sysctl.d/99-cis.conf << 'EOF'
```

```
net.ipv4.tcp_syncookies = 1
net.ipv4.conf.all.rp_filter = 1
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.all.accept_source_route = 0
kernel.randomize_va_space = 2
kernel.dmesg_restrict = 1
fs.protected_symlinks = 1
```

```
fs.protected_hardlinks = 1
EOF
```

```
sysctl --system
```

```
# — 5. fail2ban —————
cat > /etc/fail2ban/jail.local << 'EOF'
[DEFAULT]
bantime = 86400
findtime = 600
maxretry = 3
```

```
[sshd]
enabled = true
EOF
```

```
systemctl enable --now fail2ban
```

```
# — 6. Disable unused services —————
for svc in avahi-daemon cups rpcbind bluetooth; do
    systemctl disable --now "$svc" 2>/dev/null || true
done
```

```
# — 7. Secure /tmp —————
# /etc/fstab
echo "tmpfs /tmp tmpfs defaults,noexec,nosuid,nodev 0 0" >> /etc/fstab
mount -o remount /tmp
```

```
# — 8. Verify with Lynis —————
lynis audit system --quiet
# Review hardening index score
```

Topic 13: Linux Containers Internals

Namespaces, cgroups, seccomp, capabilities — The Engine Behind Docker/K8s

1. Core Concept

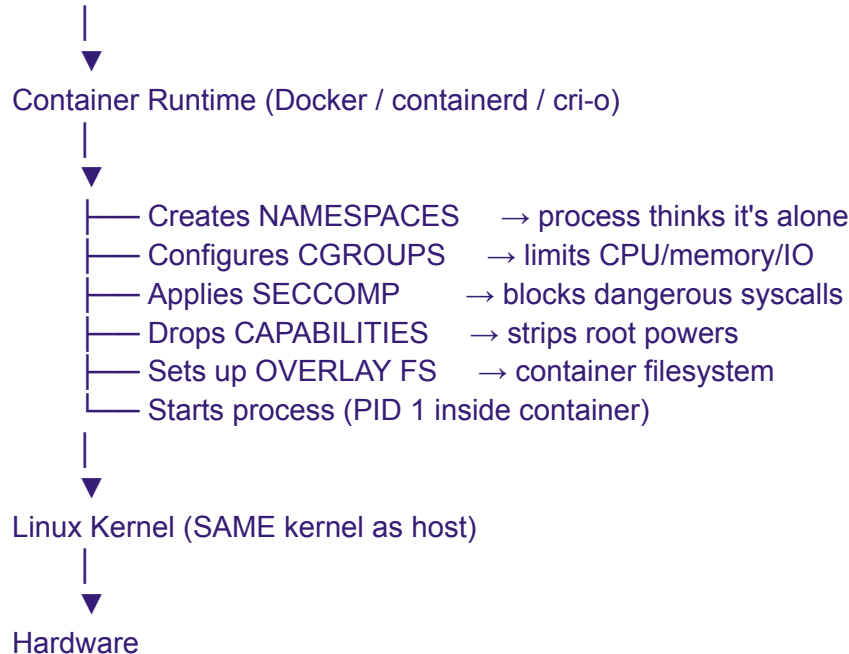
Containers are **not virtual machines** — there is no hypervisor, no separate kernel. A container is just a **Linux process** with:

- **Namespaces** → isolated view of the system (what it can SEE)
- **cgroups** → resource limits (what it can USE)
- **seccomp + capabilities** → restricted syscalls (what it can DO)

When you run `docker run nginx`, Docker calls Linux kernel APIs directly. Understanding this is what separates engineers who **use** containers from engineers who **operate** them at scale.

2. The Full Container Stack

`docker run nginx`



3. Interview Questions & Answers

Q1. What is the difference between a container and a virtual machine?

A VM runs on a **hypervisor** which emulates hardware — each VM has its **own kernel**. Full isolation but heavy (GBs of memory, minutes to start).
A container shares the **host kernel** — isolation is provided by Linux namespaces (what it sees) and cgroups (what it uses). Lightweight (MBs, milliseconds to start). Trade-off: weaker isolation boundary — a kernel vulnerability can potentially affect all containers. In production: VMs for strong isolation boundaries, containers for application density within a VM.

Q2. If a container runs as root, why can't it modify host files?

Three mechanisms prevent this:

1. **MNT namespace** — container has its own mount table, sees overlay filesystem as its `/`, not the host's `/`
2. **Capabilities** — Docker drops dangerous capabilities like `CAP_SYS_ADMIN` by default, so even root inside can't do privileged kernel operations
3. **seccomp** — dangerous syscalls like `mount`, `reboot` are blocked at kernel level

But if someone runs `docker run --privileged` or bind-mounts `/` — these protections collapse. `--privileged` is equivalent to root on the host.

Q3. Container is getting OOM killed. How do you diagnose and fix?

bash

Step 1: Confirm OOM kill

`docker inspect mycontainer | grep OOMKilled`

"OOMKilled": true

Step 2: Check configured limit

`docker inspect mycontainer | grep -i memory`

"Memory": 536870912 ← 512MB limit

Step 3: Check actual usage at time of kill

(need monitoring — Prometheus, cAdvisor)

Or check cgroup stats:

`cat /sys/fs/cgroup/docker/<id>/memory.events`

oom_kill 3 ← killed 3 times

Step 4: Check what's using memory inside container

`docker stats mycontainer` # Live memory usage

`docker exec mycontainer cat /proc/meminfo`

Step 5: Fixes:

Option A: Increase limit (if app legitimately needs more)

`docker run --memory=1g myapp`

Option B: Fix memory leak in application

Java: analyze heap dump, tune `-Xmx`

Go: use pprof

Python: use `memory_profiler`

Step 6: Add memory monitoring alert

Alert at 80% of limit — before OOM

Q4. How does Kubernetes use cgroups?

Every **Pod** in Kubernetes gets a cgroup hierarchy. When you set `resources.requests` and `resources.limits` in a Pod spec, Kubernetes translates these to cgroup settings:

- `cpu.request` → `cpu.shares` (relative weight for CPU scheduling)
- `cpu.limit` → `cpu.max` (quota/period, enforced via CFS bandwidth control)
- `memory.limit` → `memory.max` (hard limit, OOM kill if exceeded)

QoS classes map to cgroup priority:

- **Guaranteed** (requests=limits) → highest priority
- **Burstable** (requests<limits) → medium priority
- **BestEffort** (no requests/limits) → lowest priority, evicted first

CPU throttling (`nr_throttled` in `cpu.stat`) is the most common performance issue in K8s — limits set too low cause throttling even when nodes have free CPU.

Q5. What happens when you run `docker run nginx` — step by step?

1. Docker CLI sends API request to **dockerd**
 2. dockerd checks if nginx image exists locally — if not, pulls from registry
 3. dockerd calls **containerd** to create container
 4. containerd calls **runc** with OCI spec (config.json)
 5. runc calls `clone()` syscall with namespace flags → creates new **PID, NET, MNT, UTS, IPC namespaces**
 6. runc configures **cgroups** — sets CPU/memory limits
 7. runc applies **seccomp** profile and drops **capabilities**
 8. runc sets up **OverlayFS** — stacks image layers + new writable layer
 9. runc executes nginx as PID 1 inside container
 10. containerd sets up **veth pair** for networking, adds iptables NAT rules
 11. Container is running — nginx thinks it owns the whole system
-

4. Real-World Scenario

Scenario: Kubernetes pod randomly dying with exit code 137. Happens under load. Node has plenty of free memory.

bash

Exit code 137 = 128 + 9 (SIGKILL) = OOM kill

Step 1: Confirm OOM

kubectl describe pod myapp-xxx

Last State: Terminated

Reason: OOMKilled

Exit Code: 137

Step 2: Check memory limits

kubectl get pod myapp-xxx -o yaml | `grep` -A10 resources

limits:

memory: 256Mi ← limit set

requests:

memory: 128Mi

Step 3: Check actual usage trend

kubectl `top` pod myapp-xxx # Current usage

Need historical: check Prometheus/Grafana

Step 4: Dig deeper — what's inside the container?

```
kubectl exec -it myapp-xxx -- cat /proc/meminfo
```

```
kubectl exec -it myapp-xxx -- top
```

Step 5: Check cgroup directly on node

SSH to node

```
NODE=$(kubectl get pod myapp-xxx -o jsonpath='{.spec.nodeName}')
```

```
ssh $NODE
```

Find container's cgroup

```
CONTAINER_ID=$(kubectl inspect pod myapp-xxx -o
```

```
jsonpath='{.status.containerStatuses[0].containerID}' | cut -d/ -f3)
```

```
cat /sys/fs/cgroup/kubepods/.../memory.events
```

oom_kill count increasing

Step 6: Immediate fix — increase limit

```
kubectl patch deployment myapp -p \
```

```
'{"spec":{"template":{"spec":{"containers":[{"name":"myapp","resources":{"limits":{"memory":"512Mi"}}}]}}}}'
```

Step 7: Root cause

App under load was caching aggressively

Java heap not bounded (-Xmx not set → defaults to 25% of container memory)

Container limit = 256Mi, JVM heap trying for 25% of HOST memory

Fix: set JVM flags

```
# JAVA_OPTS="-Xms128m -Xmx200m"
```

```
# OR use JVM container awareness (Java 11+):
```

```
# -XX:MaxRAMPercentage=75.0
```

Step 8: Long term

Set memory alerts at 80% of limit

Use VPA (Vertical Pod Autoscaler) to auto-tune limits.

Topic 14: Cron & Job Scheduling

crontab, anacron, at, systemd timers — Complete Scheduling Playbook

1. Core Concept

Job scheduling in Linux has **four tools** — each solves a different problem. Picking the wrong one is a common mistake that causes missed jobs, duplicate runs, and silent failures. Interviews test whether you know **which tool to use when** — not just crontab syntax.

2. The Four Scheduling Tools — When to Use Which

Tool	Use When
cron	Recurring jobs on ALWAYS-ON servers Exact time required (9am, every 5min)
anacron	Recurring jobs on servers that may be OFF Daily/weekly/monthly — exact time NOT needed
at	ONE-TIME job at a specific future time "Run this once, tonight at 2am"
systemd timers	Modern replacement for cron Need logging, dependencies, missed-run catch

Key difference — cron vs anacron:

cron: "Run at 2am" → if server is off at 2am, job is MISSED forever

anacron: "Run daily" → if server was off, runs the job when it comes back

3. Interview Questions & Answers

Q1. What is the difference between cron and anacron?

cron is time-based — it runs jobs at exact times. If the server is off at the scheduled time, **the job is permanently missed**. Best for servers that run 24/7.

anacron is interval-based — it tracks "was this run in the last N days?" and runs it on next boot if missed. It **cannot schedule jobs more frequently than daily**. Best for laptops, desktops, or servers that may be rebooted.

In practice: `/etc/cron.daily/`, `/etc/cron.weekly/`,

`/etc/cron.monthly/` are run by **both** — cron runs them at exact times, anacron catches any that were missed.

Q2. cron job is not running — how do you troubleshoot?

bash

Step 1: Is crond running?

systemctl status crond # RHEL

systemctl status cron # Ubuntu

Step 2: Check cron logs

```
journalctl -u crond --since today
grep CRON /var/log/syslog
```

Step 3: Verify syntax

```
crontab -l          # Is entry there?
```

Use crontab validator online or:

```
# Run: echo "* * * * * /path/to/script" | crontab -
```

Step 4: Test script manually AS CRON WOULD

Cron runs with minimal env — simulate it:

```
env -i HOME=/root LOGNAME=root \
  PATH=/usr/bin:/bin \
  SHELL=/bin/sh \
  /opt/scripts/job.sh
```

Step 5: Check permissions

```
ls -la /opt/scripts/job.sh  # Is it executable?
```

```
chmod +x /opt/scripts/job.sh
```

Step 6: Check output capture

If no >> /logfile 2>&1, output goes to email or is lost

Add output redirection temporarily

Step 7: Check cron.allow/cron.deny

```
cat /etc/cron.allow      # Is user listed?
```

Step 8: Check if script uses relative paths

Cron runs from \$HOME, not script directory

Fix: add cd /opt/scripts/ && at start of command

Q3. How do you prevent a cron job from running twice simultaneously?

Use **flock** — it's atomic, handles crashes cleanly:

```
bash
```

In crontab:

```
* /5 * * * * flock -n /tmp/myjob.lock /opt/scripts/job.sh
```

flock -n tries to acquire a lock on the file. If another instance already holds it, flock exits immediately (skips this run). When the job finishes (or crashes), the OS automatically releases the lock. No manual cleanup needed.

For systemd timers: **Type=oneshot** prevents parallel runs automatically — systemd won't activate the service if the previous instance is still running.

Q4. What is @reboot in crontab and what are its limitations?

@reboot runs a command **once at system startup**, similar to a startup script.

Useful for: starting user-space daemons, initializing temp files, sending boot notifications.

Limitations:

- Runs **after** crond starts, which may be before all services are ready (no dependency management)
- No guarantee of network availability, mounted filesystems, etc.
- No retry on failure
- Hard to debug (no easy log capture)

For production startup tasks, **systemd units are superior** — you can specify `After=network.target`, set `Restart=on-failure`, and logs go to journald automatically.

Q5. When would you use `at` over cron?

`at` is for **one-time future execution** — use it when:

- **Maintenance windows:** `echo "systemctl stop myapp" | at 2:00 AM` — doesn't leave a recurring entry
- **Deferred dangerous commands:** schedule a config reload 5 minutes later, giving time to safely disconnect
- **One-off data migrations:** run once after a deploy, not recurring
- **Delayed notifications:** remind the team of something at a specific time

Never use `at` for recurring jobs — it doesn't reschedule itself after running. That's cron's job.

4. Real-World Scenario

Scenario: Team needs three scheduled jobs:

1. Database backup — daily at 2am, must catch missed runs if server reboots
2. API health check — every 5 minutes, must not overlap if check hangs
3. One-time database migration — tonight at midnight, never to recur

bash

— Job 1: DB Backup — systemd timer (catches missed runs) —————

/etc/systemd/system/db-backup.service

`cat` > /etc/systemd/system/db-backup.service << 'EOF'

[Unit]

Description=Daily Database Backup

After=postgresql.service

Requires=postgresql.service

[Service]

Type=oneshot

User=postgres

ExecStart=/opt/scripts/db-backup.sh

StandardOutput=journal

StandardError=journal

EOF

```

# /etc/systemd/system/db-backup.timer
cat > /etc/systemd/system/db-backup.timer << 'EOF'
[Unit]
Description=Daily DB Backup at 2am

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true          # Catches missed runs on reboot
RandomizedDelaySec=60    # Slight random delay

[Install]
WantedBy=timers.target
EOF

systemctl daemon-reload
systemctl enable --now db-backup.timer

# Verify next run:
systemctl list-timers db-backup.timer

# — Job 2: API Health Check — cron with flock (no overlap) —————

# /etc/cron.d/api-healthcheck
cat > /etc/cron.d/api-healthcheck << 'EOF'
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=ops@company.com

# flock prevents overlap if check hangs
*/5 * * * * deploy flock -n /tmp/healthcheck.lock \
    timeout 240 /opt/scripts/healthcheck.sh \
    >> /var/log/healthcheck.log 2>&1
EOF

# timeout 240 = kill after 4min (won't block next 5min run)
# flock -n = skip if already running

# — Job 3: One-time DB migration — at command —————

# Write migration script
cat > /tmp/run-migration.sh << 'EOF'
#!/bin/bash
set -euo pipefail
LOG="/var/log/migration-$(date +%Y%m%d-%H%M%S).log"
exec >> "$LOG" 2>&1

```

```
echo "Starting migration at $(date)"
/opt/scripts/migrate.sh
echo "Migration complete at $(date)"
EOF
chmod +x /tmp/run-migration.sh

# Schedule for midnight tonight
echo "/tmp/run-migration.sh" | at midnight
# Verify it's scheduled
atq
# 5 Tue Jan 16 00:00:00 2024 a root

# View job contents
at -c 5

# Monitor when it runs
tail -f /var/log/migration-*.log

# — Verification of all three —————

# Job 1: systemd timer
systemctl list-timers db-backup.timer

# Job 2: cron
crontab -u deploy -l
tail -f /var/log/healthcheck.log

# Job 3: at queue
atq
```

-Cheshta Jain

<https://www.linkedin.com/in/cheshta-j/>